

Preparazione alla discussione del codice

Free Walking Tour Siracusa

Guida funzionale, riferimenti puntuali e matrice completa di copertura dei sorgenti.

Stato del progetto: 1 luglio 2026

Copertura verificata: 16 file, 3.792 righe

Tecnologie: Flask, Flask-Login, SQLite, HTML5, CSS3, Bootstrap

322607 Giorgio Agnello

Indice

Indice	2
Perimetro e validità dei riferimenti	6
1. Metodo per rispondere al professore	7
2. Modello mentale dell'applicazione	8
2.1 Flusso generale di una richiesta	8
2.2 Responsabilità dei livelli	8
2.3 Perché rendering server-side	8
3. Avvio, dipendenze e configurazione	9
3.1 Dipendenze	9
3.2 Creazione dell'app	9
3.3 Costanti applicative	9
3.4 Avvio diretto	9
4. Struttura comune: head, navbar, flash e footer	10
4.1 Template base e blocchi Jinja	10
4.2 Navbar desktop	10
4.3 Ricerca dalla navbar	10
4.4 Avatar con iniziali	10
4.5 Navbar mobile	10
4.6 Messaggi flash	11
4.7 Footer	11
5. Homepage	12
5.1 Route e Top 3	12
5.2 Introduzione e "How it works"	12
5.3 Card dinamiche	12
6. Pagina Tours e filtri	13
6.1 Form dei filtri	13
6.2 Validazione dei filtri	13
6.3 Applicazione dei filtri	13
6.4 Card e pannello informazioni	13
7. Registrazione, login, sessione e logout	14
7.1 Registrazione frontend	14
7.2 Registrazione backend	14
7.3 Login frontend	14
7.4 Login backend	14
7.5 Ricostruzione dell'utente	14

7.6 Logout	14
8. Pianificazione di un tour	15
8.1 Autorizzazione	15
8.2 Form condiviso fra creazione e modifica	15
8.3 Campi principali	15
8.4 Schedule settimanale	15
8.5 Tappe e descrizione	15
8.6 Upload multiplo	15
8.7 Validazione completa e overlap guida	15
8.8 Scrittura del tour	16
9. Modifica del tour	17
9.1 Controlli preliminari	17
9.2 Precompilazione	17
9.3 Rimozione e aggiunta foto	17
9.4 Aggiornamento transazionale	17
10. Dettaglio del tour	18
10.1 Preparazione backend	18
10.2 Intestazione, like e stato editing	18
10.3 Hero, descrizione e tappe	18
10.4 Galleria e modal	18
11. Like e commenti	19
11.1 Like toggle	19
11.2 Commenti	19
11.3 Badge e avatar	19
12. Agenda e prenotazione	20
12.1 Generazione delle date	20
12.2 Interfaccia in base al ruolo	20
12.3 Controlli JavaScript	20
12.4 Controlli backend della prenotazione	20
12.5 Overlap partecipante	20
12.6 Nuova prenotazione o riattivazione	20
12.7 Calcolo capienza	21
13. Profilo partecipante e cancellazione	22
13.1 Smistamento profilo	22
13.2 Preparazione del profilo partecipante	22
13.3 Template	22
13.4 Cancellazione backend	22
14. Profilo guida e report	23

14.1 Tour della guida	23
14.2 Prenotazioni raggruppate per data	23
14.3 Template guida	23
14.4 Invio report	23
15. Dizionario degli helper backend	24
15.1 Autenticazione e contesto	24
15.2 Validazione e file	24
15.3 Tempo e intervalli	24
15.4 Lettura e arricchimento tour	25
15.5 Disponibilità e agende	25
15.6 Parsing e persistenza del form tour	25
16. Dizionario delle route	26
17. Database e collegamento al codice	27
17.1 Connessione	27
17.2 Reset volontario	27
17.3 Tabelle e relazioni	27
17.4 Perché ON DELETE CASCADE	27
17.5 Query parametrizzate	27
17.6 Transazioni	27
17.7 Scelte di normalizzazione	27
18. JavaScript: cosa fa e cosa non fa	28
18.1 Registrazione	28
18.2 Form tour	28
18.3 Dettaglio tour	28
18.4 Principio di sicurezza	28
19. CSS e responsive design	29
19.1 Variabili e base	29
19.2 Navbar	29
19.3 Form e upload	29
19.4 Homepage, card e profili	29
19.5 Pagina Tours	29
19.6 Dettaglio tour	29
20. Tracce complete da saper raccontare	30
20.1 Dalla navbar alla ricerca	30
20.2 Dalla registrazione al login	30
20.3 Dalla pianificazione al dettaglio	30
20.4 Dal click Book al profilo	30
20.5 Dalla cancellazione alla nuova disponibilità	30

20.6 Dal tour passato al report	30
21. Domande insidiose e risposte oneste	32
Perché sqlite3.Row?	32
Perché niente ORM?	32
Perché alcuni filtri sono in Python?	32
Perché frontend e backend duplicano alcune regole?	32
Perché password hash e non cifratura reversibile?	32
Perché una guida non può prenotare?	32
Perché l'editing è tutto bloccato?	32
Le prenotazioni passate bloccano l'edit?	32
Perché non esiste una tabella delle singole uscite?	32
Come viene impedito il doppio booking?	32
È presente CSRF protection?	32
La validazione email è completa?	32
Perché debug=True?	33
22. Matrice di copertura completa	34
22.1 app.py - 1.321 righe	35
22.2 db.py - 26 righe	36
22.3 schema.sql - 109 righe	36
22.4 requirements.txt - 3 righe	36
22.5 templates/base.html - 151 righe	36
22.6 templates/index.html - 102 righe	37
22.7 templates/tours.html - 113 righe	37
22.8 templates/login.html - 41 righe	37
22.9 templates/register.html - 88 righe	37
22.10 templates/create_tour.html - 278 righe	38
22.11 templates/tour_detail.html - 351 righe	38
22.12 templates/participant_profile.html - 71 righe	38
22.13 templates/guide_profile.html - 128 righe	39
22.14 static/assets/css/style.css - 544 righe	39
22.15 static/assets/css/tours.css - 165 righe	39
22.16 static/assets/css/tour-detail.css - 301 righe	40
23. Checklist finale per la preparazione	41

Questa guida serve per prepararsi a domande del tipo:

- Che cosa fa questa parte di codice?
- Perché esiste?
- Da dove arrivano i dati mostrati?
- Quale file riceve il form?
- Chi impedisce un'operazione non valida?
- Perché è stata scelta questa soluzione?
- Che cosa cambieresti in un progetto più grande?

La guida segue prima l'esperienza dell'utente, iniziando dagli elementi comuni a tutte le pagine. In seguito analizza helper, route, database, JavaScript e CSS. L'ultima parte contiene una matrice di copertura completa del codice sorgente.

Perimetro e validità dei riferimenti

I numeri di riga sono riferiti allo stato del progetto del **1 luglio 2026**. Sono coperti tutti i file testuali che partecipano all'esecuzione dell'app:

- `app.py`, `db.py`, `schema.sql` e `requirements.txt`;
- tutti i file `.html` dentro `templates/`;
- tutti i file `.css` dentro `static/assets/css/`.

In totale sono referenziate **3.792 righe**. Gli intervalli della matrice finale sono continui: includono anche righe vuote, commenti e tag di chiusura.

Non sono numerati:

- `database.db`, perché è un file binario di dati;
- immagini e upload, perché non sono codice;
- il PDF della consegna;
- README e guide precedenti, perché sono documentazione e non codice eseguito;
- `.gitignore` e metadati Git.

Se un file sorgente viene modificato dopo questa data, i riferimenti successivi alla modifica possono spostarsi. In quel caso va rigenerata la matrice.

1. Metodo per rispondere al professore

Quando viene indicato un blocco di codice, conviene rispondere in questo ordine:

- 1 **Contesto:** dire in quale livello si trova: template, route, helper, query, schema, JavaScript o CSS.
- 2 **Input:** spiegare da dove arrivano i valori usati.
- 3 **Elaborazione:** descrivere la regola applicata, senza leggere il codice parola per parola.
- 4 **Output:** dire cosa viene restituito, salvato o mostrato.
- 5 **Motivazione:** collegare il blocco a un requisito o a una scelta di progettazione.
- 6 **Difesa:** indicare se esiste un secondo controllo nel backend o nel DB.

Esempio per `available_places`:

È un helper backend. Riceve connessione, tour e data specifica. Somma i posti delle sole prenotazioni attive per quella data e sottrae il totale dalla capienza del tour. Esiste perché il tour è ricorrente: la disponibilità deve essere calcolata per uscita, non globalmente. Il controllo viene ripetuto al momento del POST, quindi non dipende dal numero mostrato nel browser.

Riferimenti: `app.py`:288-301, `app.py`:992-1016, `schema.sql`:65-78.

2. Modello mentale dell'applicazione

2.1 Flusso generale di una richiesta

- 1 Il browser richiede una URL.
- 2 Flask associa la URL a una route tramite `@app.route`.
- 3 La route legge parametri GET, dati POST, file e utente autenticato.
- 4 Gli helper validano i dati ed eseguono query SQLite parametrizzate.
- 5 Una lettura termina con `render_template`; una scrittura usa `commit` e normalmente esegue un redirect.
- 6 Jinja genera HTML usando i dati passati dalla route.
- 7 Bootstrap e i CSS definiscono layout e aspetto.
- 8 Il JavaScript locale migliora l'interazione, ma il backend ripete i controlli importanti.

Riferimenti generali: `app.py:1-1321`, `db.py:9-13`, `templates/base.html:1-151`.

2.2 Responsabilità dei livelli

Livello	Responsabilità	Non deve essere l'unica difesa per
HTML/Jinja	Struttura, form, contenuti dinamici, visibilità dei controlli	Permessi e integrità dei dati
JavaScript	Feedback immediato e form più comodi	Capienza, ruolo, overlap, ownership
Flask	Autorizzazione, validazione, flussi e transazioni	Vincoli relazionali definitivi
SQLite	Persistenza, foreign key, unicità e CHECK	Messaggi di errore amichevoli
CSS/Bootstrap	Presentazione e responsive design	Regole applicative

La frase chiave per l'orale è: **il frontend aiuta l'utente, il backend decide, il database protegge l'integrità.**

2.3 Perché rendering server-side

Flask passa strutture Python ai template Jinja, che producono pagine HTML già complete. È una soluzione coerente con il corso e mantiene nello stesso flusso sessione, ruoli, query e rendering.

Alternativa: frontend separato con API JSON. Non è stata scelta perché avrebbe richiesto gestione dello stato client, richieste `fetch`, API e autenticazione separate senza un vantaggio necessario per la dimensione del progetto.

Riferimenti: `app.py:585-620`, `app.py:912-945`, `templates/index.html:1-102`, `templates/tour_detail.html:1-351`.

3. Avvio, dipendenze e configurazione

3.1 Dipendenze

`requirements.txt` dichiara Flask, Flask-Login e Werkzeug. Flask gestisce server, route, request, template, flash e redirect. Flask-Login gestisce sessione e `current_user`. Werkzeug fornisce hash delle password e pulizia dei nomi file.

Riferimenti: `requirements.txt`:1-3, `app.py`:5-9.

3.2 Creazione dell'app

`Flask(__name__)` crea l'applicazione. La `SECRET_KEY` firma cookie di sessione e messaggi flash; viene letta dall'ambiente e ha un fallback solo per sviluppo. `BASE_DIR` rende i percorsi indipendenti dalla cartella da cui parte il comando. La cartella `upload` viene creata se non esiste.

Riferimenti: `app.py`:14-24.

Possibile domanda: **Perché non lasciare la secret key fissa in produzione?**

Risposta: perché chi la conosce potrebbe falsificare dati firmati della sessione. In deploy va impostata come variabile d'ambiente.

3.3 Costanti applicative

Le lingue, i weekday, i filtri di durata e il minimo di quattro tappe sono costanti centrali. Il `context_processor` le rende disponibili a tutti i template senza passarle manualmente da ogni route.

Riferimenti: `app.py`:24-42, `app.py`:81-89.

Possibile domanda: **Perché le lingue non sono una tabella?**

Risposta: sono cinque valori fissi stabiliti dal dominio. Una tabella avrebbe aggiunto query e gestione senza necessità. In un sistema amministrabile o multilingua sarebbe invece ragionevole normalizzarle.

3.4 Avvio diretto

Il blocco `if __name__ == "__main__"` avvia il server solo quando `app.py` è eseguito direttamente. `host="0.0.0.0"` accetta connessioni sulle interfacce di rete; `debug=True` è utile in sviluppo ma non deve essere il server di produzione.

Riferimento: `app.py`:1320-1321.

4. Struttura comune: head, navbar, flash e footer

4.1 Template base e blocchi Jinja

Tutte le pagine estendono `base.html`. Il template definisce:

- lingua del documento, charset e viewport;
- titolo sostituibile con `{% block title %}`;
- favicon;
- Bootstrap, Bootstrap Icons e CSS globale;
- blocco `extra_css` per CSS specifici;
- blocco `content` per il contenuto della pagina;
- blocco `scripts` per JavaScript specifico.

Questa eredità evita di duplicare navbar, footer e dipendenze.

Riferimenti: `templates/base.html:1-15`, `templates/base.html:132-151`.

4.2 Navbar desktop

La navbar desktop è una griglia a tre zone:

- 1 Home e ricerca a sinistra;
- 2 nome del sito al centro;
- 3 Tours, eventuale Plan e autenticazione a destra.

Jinja usa `current_user.is_authenticated` e `current_user.role` per mostrare:

- Sign in e Register ai visitatori;
- iniziali, profilo e logout agli autenticati;
- Plan e Plan tour soltanto alle guide.

Nascondere un link non è una misura di sicurezza: le route protette ripetono il controllo con Flask-Login e `require_role`.

Riferimenti frontend: `templates/base.html:16-70`.

Riferimenti backend: `app.py:43-45`, `app.py:193-199`, `app.py:727-732`, `app.py:1101-1104`.

Riferimenti stile: `static/assets/css/style.css:25-114`.

4.3 Ricerca dalla navbar

Il form usa metodo GET e invia q a `/tours`. Il valore compare nella query string e viene gestito dalla stessa route dei filtri. Non serve una route di ricerca separata.

Riferimenti: `templates/base.html:20-28`, `app.py:599-620`, `app.py:400-431`.

Perché GET: la ricerca non modifica dati, l'URL resta leggibile e ricaricabile.

4.4 Avatar con iniziali

L'avatar non dipende da un'immagine caricata: usa la prima lettera di nome e cognome dell'utente presente in sessione. I dati arrivano dall'oggetto User ricostruito da `load_user`.

Riferimenti: `templates/base.html:38-61`, `app.py:50-78`, `static/assets/css/style.css:68-80`.

4.5 Navbar mobile

Sotto il breakpoint Bootstrap lg, la navbar desktop viene sostituita da una topbar e da un pannello collapse. Il menu contiene link equivalenti e una ricerca centrata. Bootstrap gestisce apertura e chiusura usando attributi `data-bs-*`; non serve JavaScript custom.

Riferimenti: `templates/base.html:71-117`, `static/assets/css/style.css:115-145`, `static/assets/css/style.css:207-230`.

4.6 Messaggi flash

Le route chiamano `flash(messaggio, categoria)`. `base.html` legge tutti i messaggi e converte la categoria in una classe Bootstrap come `alert-success`, `alert-danger` o `alert-warning`. Il pulsante di chiusura usa il bundle JS di Bootstrap.

Riferimenti: `templates/base.html:119-130`, `templates/base.html:148-149`.

Esempi backend: `app.py:632-655`, `app.py:687-714`, `app.py:1057-1063`.

4.7 Footer

Il footer contiene disclaimer, attribuzione, contatto e autore. I link esterni usano `target="_blank"` e `rel="noopener noreferrer"`. Il body è una colonna flex alta almeno quanto la viewport e il footer usa `margin-top:auto`: così resta in fondo nelle pagine corte senza coprire le pagine lunghe.

Riferimenti: `templates/base.html:134-147`, `static/assets/css/style.css:10-19`, `static/assets/css/style.css:472-506`, `static/assets/css/style.css:534-543`.

5. Homepage

5.1 Route e Top 3

La route / apre il DB, ottiene tutti i tour arricchiti, li ordina in ordine decrescente per like, commenti e id, prende i primi tre e chiude la connessione. L'id rende stabile il criterio in caso di parità.

Riferimenti backend: `app.py:587-596`, `app.py:400-431`, `app.py:385-397`.

Riferimenti database: `schema.sql:25-38`, `schema.sql:80-99`.

5.2 Introduzione e “How it works”

La prima parte è statica e comunica il dominio. Le tre card spiegano visibilità pubblica, prenotazioni per data e lingue. Il link `View all` collega la homepage alla lista completa.

Riferimenti: `templates/index.html:1-59`, `static/assets/css/style.css:147-191`.

5.3 Card dinamiche

Il ciclo Jinja riceve tours dalla route. Ogni elemento usa foto principale, lingua, tema, guida, durata, schedule, like e commenti. `{% else %}` del ciclo gestisce il database senza tour.

Riferimenti: `templates/index.html:60-102`, `app.py:221-244`, `app.py:247-283`, `app.py:385-397`, `static/assets/css/style.css:368-414`.

Domanda: **Perché arricchire i tour in Python?**

Risposta: la riga base resta semplice, mentre schedule, foto e contatori vengono aggiunti da helper riusabili. È leggibile per un progetto piccolo. Con molti tour sarebbe più efficiente aggregare i contatori in SQL ed evitare più query per riga.

6. Pagina Tours e filtri

6.1 Form dei filtri

Il form usa GET e contiene testo, data, fascia di durata e lingua. Lingue e fasce sono generate dalle costanti iniettate da Flask. Apply mantiene i valori selezionati; Reset torna a /tours senza query string.

Riferimenti frontend: `templates/tours.html:1-78`.

Riferimenti backend: `app.py:36-40`, `app.py:81-89`, `app.py:599-620`.

6.2 Validazione dei filtri

La route accetta soltanto chiavi durata e lingue note. Una data non ISO genera `ValueError`, viene rimossa e produce un messaggio flash. Questo impedisce che parametri URL manipolati rompano la pagina.

Riferimenti: `app.py:599-620`, `app.py:166-170`.

6.3 Applicazione dei filtri

`filtered_tours` legge tour e guide con una JOIN, arricchisce ogni tour e applica:

- ricerca case-insensitive su titolo o tema;
- uguaglianza della lingua;
- intervallo di durata;
- presenza di uno schedule nel weekday della data scelta.

Se è scelta una data, `selected_seats_left` permette alla card di mostrare posti rimasti per quel giorno.

Riferimenti: `app.py:400-431`, `app.py:385-397`, `templates/tours.html:79-113`.

Nota da sapere: il filtro data verifica che il tour sia normalmente previsto in quel weekday; non limita il risultato ai 60 giorni mostrati nell'agenda.

6.4 Card e pannello informazioni

La card intera è un link. Il CSS sovrappone un gradiente per rendere bianco il titolo e mostra un pannello dettagli al passaggio del mouse. Su mobile il pannello si muove verticalmente e la card diventa più alta.

Riferimenti: `templates/tours.html:79-113`, `static/assets/css/tours.css:41-165`.

7. Registrazione, login, sessione e logout

7.1 Registrazione frontend

Il form raccoglie nome, cognome, email, password e ruolo. Il pannello lingue è nascosto per default e viene mostrato da JavaScript quando il ruolo è guide. `setCustomValidity` richiede almeno una lingua alla guida.

Riferimenti: `templates/register.html:1-69`, `templates/register.html:71-88`.

7.2 Registrazione backend

La route normalizza l'email, pulisce i testi, valida lunghezze, formato minimo dell'email, password, ruolo e lingue. Se ci sono errori usa flash e non scrive. La password viene trasformata in hash PBKDF2 prima dell'INSERT. Il vincolo SQL `UNIQUE(email)` è la difesa finale contro duplicati.

Riferimenti: `app.py:658-715`, `app.py:94-95`, `schema.sql:13-23`.

Domanda: **Perché controllare email unica sia in app sia nel DB?**

Risposta: l'app produce un messaggio comprensibile; il vincolo DB garantisce l'integrità anche con richieste concorrenti o codice futuro.

7.3 Login frontend

Il form invia email, password e ruolo. Se esiste next nella query string viene preservato nell'action, così dopo il login l'utente torna alla pagina protetta.

Riferimenti: `templates/login.html:1-41`.

7.4 Login backend

La route cerca email e ruolo con query parametrizzata. Solo dopo verifica l'hash della password. Se è corretto costruisce User, chiama `login_user` e usa `safe_redirect`.

Riferimenti: `app.py:625-655`, `app.py:186-190`.

`safe_redirect` accetta solo percorsi che iniziano con una singola `/`; evita un open redirect verso un sito esterno.

7.5 Ricostruzione dell'utente

Flask-Login salva l'id nella sessione. A ogni richiesta autenticata `load_user` legge la riga aggiornata e ricrea User. La proprietà `spoken_languages` trasforma la stringa separata da virgole in lista.

Riferimenti: `app.py:48-78`, `db.py:9-13`.

7.6 Logout

`@login_required` impedisce logout senza sessione. `logout_user` rimuove i dati della sessione e il redirect torna alla homepage.

Riferimento: `app.py:718-722`.

Nota critica: in un'app di produzione il logout sarebbe preferibilmente POST con protezione CSRF. Qui è una route GET semplice, coerente con lo scope didattico.

8. Pianificazione di un tour

8.1 Autorizzazione

La route richiede login e ruolo guida. Il controllo server-side esiste anche se il link Plan è nascosto ai partecipanti.

Riferimenti: `app.py:727-732`, `app.py:193-199`, `templates/base.html:34-36`.

8.2 Form condiviso fra creazione e modifica

`create_tour.html` usa la presenza di `tour` per cambiare titolo, action, valori iniziali e testo del pulsante. Una sola struttura evita due form quasi identici.

Riferimenti: `templates/create_tour.html:1-17`, `templates/create_tour.html:120-128`.

8.3 Campi principali

Il form contiene titolo, tema, meeting point, lingua, durata e capienza. Il menu lingua usa soltanto `current_user.spoken_languages`. Gli attributi HTML `minlength`, `min`, `max` e `required` danno feedback immediato.

Riferimenti frontend: `templates/create_tour.html:17-54`.

Riferimenti backend: `app.py:456-493`.

8.4 Schedule settimanale

Ogni weekday ha checkbox e input time. JavaScript rende obbligatorio e attivo l'orario solo quando il giorno è selezionato. Il backend scorre gli stessi nomi `day_0`, `time_0`, ecc., controlla formato e richiede almeno un giorno.

Riferimenti frontend: `templates/create_tour.html:56-71`, `templates/create_tour.html:130-147`.

Riferimenti backend: `app.py:436-453`.

Riferimento DB: `schema.sql:39-46`.

8.5 Tappe e descrizione

Le tappe possono essere separate da virgole o righe. JavaScript e backend usano la stessa idea di parsing; entrambi richiedono almeno quattro tappe. La descrizione richiede almeno 30 caratteri.

Riferimenti frontend: `templates/create_tour.html:73-82`, `templates/create_tour.html:148-175`.

Riferimenti backend: `app.py:177-179`, `app.py:456-476`.

Riferimenti DB: `schema.sql:48-55`.

8.6 Upload multiplo

L'input file è nascosto e ha `multiple`. JavaScript mostra i nomi, permette di rimuovere un file prima dell'invio usando `DataTransfer` e imposta un errore se in creazione ci sono meno di cinque immagini.

Riferimenti frontend: `templates/create_tour.html:84-119`, `templates/create_tour.html:176-258`.

Riferimenti backend: `app.py:98-113`, `app.py:528-538`, `app.py:566-582`.

8.7 Validazione completa e overlap guida

`parse_tour_form` produce una struttura dati e una lista errori. Per ogni slot chiama `guide_has_overlap`, che legge gli altri tour dello stesso weekday, converte gli orari in minuti e confronta intervalli semiaperti.

Formula dell'overlap:

```
startA < endB AND startB < endA
```

Due tour possono quindi essere consecutivi: se uno termina esattamente quando l'altro inizia, non si sovrappongono.

Riferimenti: `app.py:140-163`, `app.py:316-333`, `app.py:456-525`.

8.8 Scrittura del tour

La route inserisce prima la riga `tours`, recupera `lastrowid`, poi inserisce `schedule`, `tappe` e `foto` nelle tabelle figlie. Il commit avviene alla fine. In caso di `ValueError` viene eseguito `rollback`.

Riferimenti: `app.py:734-778`, `app.py:566-582`, `schema.sql:25-64`.

Perché più tabelle: `schedule`, `tappe` e `foto` sono relazioni uno-a-molti; salvarle in campi testuali renderebbe difficili ordinamento, filtri e vincoli.

9. Modifica del tour

9.1 Controlli preliminari

La route richiede guida, tour esistente, ownership e assenza di prenotazioni attive. 404 indica risorsa inesistente; 403 indica risorsa esistente ma non autorizzata.

Riferimenti: `app.py:781-800`, `app.py:304-313`.

Importante: `tour_has_active_reservations` cerca qualunque riga con stato booked, anche relativa a una data passata. Una prenotazione cancellata non blocca; una passata ancora booked sì.

9.2 Precompilazione

Schedule, tappe e foto vengono letti dal DB e convertiti nella forma attesa dal template: dizionario weekday-orario, stringa di tappe e lista di foto.

Riferimenti: `app.py:801-806`, `app.py:204-236`.

9.3 Rimozione e aggiunta foto

Le foto esistenti sono mostrate con checkbox nascoste. Il pulsante alterna Remove/Undo. Il backend converte gli id in interi, verifica che appartengano al tour e calcola il totale finale:

```
foto correnti - foto rimosse + nuovi file
```

Se sono state richieste modifiche alle foto, il totale non può scendere sotto 5.

Riferimenti frontend: `templates/create_tour.html:91-119`, `templates/create_tour.html:176-278`.

Riferimenti backend: `app.py:541-565`, `app.py:807-838`.

9.4 Aggiornamento transazionale

La riga principale viene aggiornata. Schedule e tappe vengono eliminate e reinserite, perché sono liste piccole e ordinate. Le foto selezionate vengono eliminate, le nuove aggiunte dopo l'ultima posizione e infine tutte vengono reindicizzate.

Il DB viene confermato prima di cancellare i file fisici rimossi. Così un errore prima del commit non lascia il database con riferimenti già cancellati dal disco.

Riferimenti: `app.py:840-909`, `app.py:554-565`, `app.py:116-125`.

Alternativa: modificare ogni schedule e tappa singolarmente. Non è stata scelta perché renderebbe il form e il backend molto più complessi per liste ridotte.

10. Dettaglio del tour

10.1 Preparazione backend

La route legge tour e guida con JOIN, arricchisce il tour, carica tappe, foto, prossime date e commenti con autore. Se il tour non esiste restituisce 404.

Riferimenti: `app.py:914-945`, `app.py:273-283`, `app.py:359-397`.

10.2 Intestazione, like e stato editing

La pagina mostra dati essenziali e il form like. Se l'utente è la guida autrice, mostra Edit oppure il badge di blocco in base a `tour.can_edit` calcolato dal backend.

Riferimenti: `templates/tour_detail.html:8-50`, `static/assets/css/tour-detail.css:180-188`.

10.3 Hero, descrizione e tappe

La foto principale è il primo record ordinato per posizione. L'overlay mostra meeting point e guida. La sezione informativa usa schedule, lingua, durata, capienza e lista ordinata delle tappe.

Riferimenti: `templates/tour_detail.html:51-100`, `app.py:221-244`, `static/assets/css/tour-detail.css:1-94`.

10.4 Galleria e modal

Le miniature hanno `data-photo-index`. Un click apre il modal Bootstrap e il JavaScript sposta il carousel all'indice scelto. Il carousel non scorre da solo, supporta touch e mostra frecce solo con più di una foto.

Riferimenti HTML: `templates/tour_detail.html:101-110`, `templates/tour_detail.html:225-261`.

Riferimenti JS: `templates/tour_detail.html:265-283`.

Riferimenti CSS: `static/assets/css/tour-detail.css:95-179`.

11. Like e commenti

11.1 Like toggle

Un visitatore viene mandato al login con next verso il tour. Per un utente autenticato la route cerca la coppia tour-utente: se esiste la elimina, altrimenti la inserisce. Il vincolo SQL impedisce due like dello stesso utente sul tour.

Riferimenti: `app.py:948-974`, `app.py:263-270`, `schema.sql:80-88`, `templates/tour_detail.html:31-36`.

11.2 Commenti

Il form è mostrato solo agli autenticati, ma la route verifica comunque la sessione. Il testo viene ripulito e deve avere da 2 a 600 caratteri. La data di pubblicazione viene generata dal server.

Riferimenti frontend: `templates/tour_detail.html:112-153`.

Riferimenti backend: `app.py:1068-1094`.

Riferimento DB: `schema.sql:90-98`.

11.3 Badge e avatar

Il ruolo arriva dalla JOIN con users. Il badge Tour author non è salvato nel commento: viene calcolato confrontando `comment.user_id` con `tour.guide_id`, evitando un dato duplicato. Il colore avatar è deterministico con `user_id % 5`: varia fra utenti ma resta stabile nel tempo.

Riferimenti: `templates/tour_detail.html:129-152`, `app.py:926-935`, `static/assets/css/tour-detail.css:249-280`.

12. Agenda e prenotazione

12.1 Generazione delle date

`upcoming_dates` considera oggi e i successivi 60 giorni. Per ogni data controlla se esiste uno schedule dello stesso weekday, scarta tour già iniziati e calcola i posti rimasti sulla data specifica.

Riferimenti: `app.py:359-382`, `app.py:211-218`, `app.py:288-301`.

12.2 Interfaccia in base al ruolo

La booking card mostra:

- login e registrazione al visitatore;
- messaggio di divieto alla guida;
- form agenda al partecipante;
- stato vuoto se non ci sono date nei prossimi 60 giorni.

Le date piene hanno radio disabilitato. La prima data con posti viene selezionata automaticamente tramite un namespace Jinja.

Riferimenti: `templates/tour_detail.html:156-220`, `static/assets/css/tour-detail.css:189-248`.

12.3 Controlli JavaScript

Quando cambia la data, JavaScript legge `data-seats`, limita il campo persone al minimo tra 4 e posti disponibili e aggiorna il testo. Controlla inoltre che il numero di nomi completi sia `num_people - 1`.

Riferimenti: `templates/tour_detail.html:284-351`.

Questi controlli migliorano il form, ma possono essere aggirati: il backend li ripete tutti.

12.4 Controlli backend della prenotazione

L'ordine della route è:

- 1 sessione e ruolo participant;
- 2 esistenza del tour;
- 3 data ISO e non passata;
- 4 appartenenza della data allo schedule;
- 5 tour non iniziato;
- 6 numero persone tra 1 e 4;
- 7 quantità e formato dei guest full names;
- 8 capienza residua;
- 9 assenza di overlap nell'agenda personale;
- 10 assenza di prenotazione attiva duplicata.

Riferimenti: `app.py:977-1039`.

12.5 Overlap partecipante

La query cerca prenotazioni booked dello stesso utente e della stessa data, unisce lo schedule corrispondente al weekday e confronta gli intervalli in minuti. Le prenotazioni cancellate non bloccano l'agenda.

Riferimenti: `app.py:336-356`, `app.py:1018-1029`.

12.6 Nuova prenotazione o riattivazione

Il vincolo DB consente una sola riga per utente, tour e data. Se la riga esiste ed è cancellata, viene riattivata e aggiornata; altrimenti viene inserita. Questo mantiene lo storico senza creare duplicati.

Riferimenti: `app.py:1031-1065`, `schema.sql:65-78`.

12.7 Calcolo capienza

Sono sommate soltanto le persone delle prenotazioni con stato booked per quel tour e quella data. Una cancellazione libera immediatamente i posti.

Riferimenti: `app.py:288-301`, `app.py:1014-1016`, `schema.sql:70-77`.

13. Profilo partecipante e cancellazione

13.1 Smistamento profilo

La URL `/profile` è comune. In base al ruolo effettua redirect al profilo guida o partecipante. Così la navbar non deve conoscere due URL.

Riferimento: `app.py:1099-1104`.

13.2 Preparazione del profilo partecipante

La route verifica il ruolo, legge tutte le prenotazioni dell'utente con tour e guida, ricostruisce l'orario dal weekday e calcola `can_cancel`. La cancellazione è disponibile solo per prenotazioni attive con almeno 24 ore di anticipo.

Riferimenti: `app.py:1107-1142`.

13.3 Template

Ogni card mostra lingua, titolo cliccabile, data, ora, meeting point, guida, numero persone, accompagnatori e stato. Il form Cancel compare soltanto se `can_cancel`; il ciclo ha uno stato vuoto.

Riferimenti: `templates/participant_profile.html:1-71`, `static/assets/css/style.css:415-471`.

13.4 Cancellazione backend

Il backend non si fida di `can_cancel` mostrato nel template. Rilegge la prenotazione limitandola al proprietario, controlla stato e soglia temporale, quindi esegue un UPDATE a `cancelled` con timestamp.

Riferimenti: `app.py:1145-1183`, `schema.sql:65-78`.

Perché cancellazione logica: mantiene lo storico, libera posti filtrando solo booked e permette una successiva riattivazione della stessa riga.

14. Profilo guida e report

14.1 Tour della guida

La route legge soltanto tour con `guide_id = current_user.id`. Ogni tour viene arricchito con schedule, foto, contatori e stato editing.

Riferimenti: `app.py:1186-1208`, `app.py:385-397`.

14.2 Prenotazioni raggruppate per data

La prima query aggrega numero di prenotazioni e somma persone per data. Per ogni gruppo vengono poi letti report e dettagli dei partecipanti. Viene calcolato `can_report` se la partenza è passata e non esiste report.

Riferimenti: `app.py:1209-1255`.

Raggruppare per data è essenziale perché lo stesso tour ricorre settimanalmente, ma presenze e report riguardano una singola uscita.

14.3 Template guida

Il template mostra tour, stato editing, gruppi data, expected participants, tabella partecipanti, stato report e form di invio. Se il report esiste mostra riepilogo e link alla foto; non mostra nuovamente il form.

Riferimenti: `templates/guide_profile.html:1-128`, `static/assets/css/style.css:415-471`.

14.4 Invio report

La route controlla:

- 1 ruolo guida;
- 2 tour esistente e di proprietà;
- 3 data appartenente allo schedule;
- 4 tour già svolto;
- 5 almeno una prenotazione attiva;
- 6 report non già presente;
- 7 partecipanti effettivi tra 0 e attesi;
- 8 foto valida.

Poi salva la foto, inserisce il report e conferma la transazione.

Riferimenti: `app.py:1260-1317`, `schema.sql:100-109`.

Il vincolo `UNIQUE(tour_id, tour_date)` impedisce un secondo report anche se un controllo applicativo venisse saltato.

15. Dizionario degli helper backend

Questa sezione è pensata per domande su una funzione isolata.

15.1 Autenticazione e contesto

Funzione/blocco	Righe	Cosa fa e perché
User	app.py:50-61	Modello minimo richiesto da Flask-Login; conserva identità, ruolo e lingue.
load_user	app.py:64-78	Ricostruisce l'utente della sessione da SQLite.
inject_constants	app.py:81-89	Espone costanti a tutti i template Jinja.
safe_redirect	app.py:186-190	Mantiene il flusso next ma rifiuta URL esterni.
require_role	app.py:193-199	Centralizza controllo login/ruolo e relativi redirect.

15.2 Validazione e file

Funzione	Righe	Cosa fa e perché
normalize_email	app.py:94-95	Elimina spazi e differenze di maiuscole prima di query/INSERT.
allowed_file	app.py:98-99	Controlla l'estensione nell'insieme consentito.
validate_file	app.py:102-103	Verifica esistenza del file, nome ed estensione.
save_uploaded_file	app.py:106-113	Pulisce nome, genera UUID, salva e restituisce URL statico.
delete_uploaded_file	app.py:116-125	Elimina solo file dentro la cartella upload, evitando path traversal.
parse_positive_int	app.py:128-137	Converte interi e applica minimo/massimo con messaggi uniformi.
parse_iso_date	app.py:166-170	Converte YYYY-MM-DD in date o genera errore controllato.
split_names	app.py:177-179	Separa valori per virgola o nuova riga.
has_first_and_last_name	app.py:182-183	Richiede almeno due parole per ogni accompagnatore.

Limite da riconoscere: l'upload controlla l'estensione, non analizza realmente il contenuto binario. In produzione si aggiungerebbero controllo MIME/magic bytes, limite dimensione e scansione; per lo scope del corso è stata mantenuta una validazione semplice.

15.3 Tempo e intervalli

Funzione	Righe	Cosa fa e perché
parse_time_to_minutes	app.py:140-145	Converte HH:MM in minuti, utili per confronti matematici.
minutes_to_time_label	app.py:148-151	Torna da minuti a etichetta leggibile.
time_range_label	app.py:154-157	Produce l'intervallo usato nei messaggi overlap.
ranges_overlap	app.py:160-163	Applica la formula di sovrapposizione a due intervalli.
tour_datetime	app.py:173-174	Combina data specifica e ora settimanale.

15.4 Lettura e arricchimento tour

Funzione	Righe	Cosa fa e perché
get_schedules	app.py:204-208	Legge slot del tour ordinati per weekday.
get_schedule_for_date	app.py:211-218	Trova lo slot usando il weekday di una data reale.
format_schedule	app.py:221-222	Costruisce etichetta compatta per card e profili.
get_stops	app.py:225-229	Legge tappe ordinate per posizione.
get_photos	app.py:232-236	Legge foto ordinate per posizione.
primary_photo	app.py:239-244	Usa la prima foto o la favicon come fallback.
tour_like_count	app.py:247-252	Conta i like del tour.
tour_comment_count	app.py:255-260	Conta i commenti del tour.
current_user_liked	app.py:263-270	Calcola lo stato del pulsante like per l'utente corrente.
get_tour_row	app.py:273-283	JOIN fra tour e guida; base comune delle route.
enrich_tour	app.py:385-397	Aggiunge dati derivati a una riga tour.

15.5 Disponibilità e agende

Funzione	Righe	Cosa fa e perché
active_reserved_places	app.py:288-297	Somma persone attive per tour e data.
available_places	app.py:300-301	Sottrae prenotati dalla capienza.
tour_has_active_reservations	app.py:304-313	Decide se l'editing generale è bloccato.
guide_has_overlap	app.py:316-333	Controlla conflitti nello schedule settimanale della guida.
participant_has_overlap	app.py:336-356	Controlla conflitti su una data reale del partecipante.
upcoming_dates	app.py:359-382	Materializza 60 giorni di date prenotabili.
filtered_tours	app.py:400-431	Applica ricerca e filtri pubblici.

15.6 Parsing e persistenza del form tour

Funzione	Righe	Cosa fa e perché
parse_schedule_form	app.py:436-453	Legge checkbox/orari e produce slot validati.
parse_tour_form	app.py:456-525	Valida tutti i campi e controlla overlap guida.
get_form_photo_files	app.py:528-529	Estrae solo file realmente selezionati.
validate_photo_files	app.py:532-538	Richiede 5 foto in creazione e valida le estensioni.
parse_photo_ids	app.py:541-551	Converte e deduplica gli id foto da rimuovere.
reindex_tour_photos	app.py:554-563	Ripristina posizioni consecutive dopo rimozioni.
insert_tour_details	app.py:566-582	Inserisce schedule, tappe e foto di un nuovo tour.

16. Dizionario delle route

Metodo e URL	Funzione	Righe	Accesso	Esito principale
GET /	index	app.py:587-596	Pubblico	Homepage con Top 3
GET /tours	tours	app.py:599-620	Pubblico	Lista filtrata
GET, POST /login	login	app.py:625-655	Pubblico	Form o sessione aperta
GET, POST /register	register	app.py:658-715	Pubblico	Form o account creato
GET /logout	logout	app.py:718-722	Login	Sessione chiusa
GET, POST /create-tour	create_tour	app.py:727-778	Guide	Form o nuovo tour
GET, POST /tour/<id>/edit	edit_tour	app.py:781-909	Guida autrice	Form o tour aggiornato
GET /tour/<id>	tour_detail	app.py:914-945	Pubblico	Dettaglio completo
POST /tour/<id>/like	toggle_like	app.py:948-974	Login	Like aggiunto/rimosso
POST /tour/<id>/book	book_tour	app.py:977-1065	Participant	Prenotazione/riattivazione
POST /tour/<id>/comment	add_comment	app.py:1068-1094	Login	Commento creato
GET /profile	profile	app.py:1099-1104	Login	Redirect per ruolo
GET /participant/profile	participant_profile	app.py:1107-1142	Participant	Riepilogo prenotazioni
POST /reservation/<id>/cancel	cancel_reservation	app.py:1145-1183	Proprietario participant	Cancellazione logica
GET /guide/profile	guide_profile	app.py:1186-1255	Guide	Tour, prenotazioni, report
POST /guide/report/<tour>/<date>	submit_report	app.py:1260-1317	Guida autrice	Report unico

Pattern POST-Redirect-GET: dopo una scrittura quasi tutte le route eseguono un redirect. Questo evita il reinvio del form quando l'utente aggiorna la pagina.

17. Database e collegamento al codice

17.1 Connessione

`get_db_connection` apre `database.db` usando un percorso assoluto, imposta `sqlite3.Row` per accedere alle colonne per nome e abilita le foreign key per ogni connessione.

Riferimenti: `db.py:1-13`.

Perché `PRAGMA foreign_keys = ON` ogni volta: SQLite non garantisce che sia attivo globalmente; l'impostazione appartiene alla connessione.

17.2 Reset volontario

`init_db` legge tutto `schema.sql`, lo esegue, conferma e chiude. Il blocco finale fa sì che il reset avvenga soltanto con `python db.py`, non importando `db` da `app.py`.

Riferimenti: `db.py:16-26`, `schema.sql:1-12`.

17.3 Tabelle e relazioni

Tabella	Righe schema	Scopo	Vincolo centrale
users	<code>schema.sql:13-23</code>	Account guida/participant	email unica, ruolo controllato
tours	<code>schema.sql:25-37</code>	Dati principali tour	FK alla guida
tour_schedule	<code>schema.sql:39-46</code>	Slot settimanali	un weekday per tour
tour_stops	<code>schema.sql:48-54</code>	Tappe ordinate	FK con cascade
tour_photos	<code>schema.sql:56-63</code>	Foto ordinate	posizione unica per tour
reservations	<code>schema.sql:65-78</code>	Booking su data reale	1-4 persone, booking unico
tour_likes	<code>schema.sql:80-88</code>	Relazione utente-tour	like unico
comments	<code>schema.sql:90-98</code>	Testi utente sul tour	FK a tour e utente
tour_reports	<code>schema.sql:100-109</code>	Consuntivo per uscita	report unico per tour-data

17.4 Perché ON DELETE CASCADE

Se un'entità padre viene eliminata, i record figli non devono restare orfani. Per esempio, eliminando un tour spariscono schedule, tappe, foto, prenotazioni, like, commenti e report collegati. L'app non espone attualmente una route di eliminazione, ma lo schema resta coerente anche per manutenzione futura.

Riferimenti: `schema.sql:36`, `schema.sql:44`, `schema.sql:53`, `schema.sql:61`, `schema.sql:75-76`, `schema.sql:85-86`, `schema.sql:96-97`, `schema.sql:107`.

17.5 Query parametrizzate

I dati utente non sono concatenati nel testo SQL: vengono passati con `?` e una tupla/lista di parametri. Questo separa istruzione e valori e riduce il rischio di SQL injection.

Esempi: `app.py:66-68`, `app.py:636-640`, `app.py:1031-1037`.

17.6 Transazioni

Creazione e modifica coinvolgono più tabelle. Il commit viene eseguito soltanto dopo che tutte le operazioni sono riuscite; il rollback evita stati parziali.

Riferimenti: `app.py:748-776`, `app.py:840-900`.

17.7 Scelte di normalizzazione

Sono normalizzate le informazioni ripetibili e interrogabili: schedule, tappe, foto, prenotazioni, like, commenti e report. Le lingue parlate dalla guida sono invece una stringa separata da virgole: scelta semplice per cinque valori fissi, ma in un progetto più grande sarebbe preferibile una tabella ponte `user_languages`.

18. JavaScript: cosa fa e cosa non fa

18.1 Registrazione

Mostra le lingue soltanto per guide e imposta una validazione HTML custom.

Riferimento: `templates/register.html`:71-88.

18.2 Form tour

- abilita orari dei soli giorni selezionati;
- conta le tappe;
- mostra nomi dei file;
- permette di rimuovere file dalla selezione;
- marca foto esistenti per la rimozione;
- calcola il totale finale minimo di cinque.

Riferimento: `templates/create_tour.html`:130-278.

18.3 Dettaglio tour

- porta il carousel alla foto cliccata;
- evidenzia la data selezionata;
- limita il numero persone ai posti mostrati;
- valida numero e formato dei nomi accompagnatori.

Riferimento: `templates/tour_detail.html`:265-351.

18.4 Principio di sicurezza

Il JavaScript è modificabile dall'utente e può essere disabilitato. Per questo ogni regola importante viene ripetuta in Flask. Il JS non esegue query né decide permessi.

19. CSS e responsive design

19.1 Variabili e base

Le custom properties definiscono palette, superficie, testo e ombra. Il body flex permette il footer in fondo. Il font usa una system stack e non richiede download esterni.

Riferimenti: `static/assets/css/style.css:1-24`.

19.2 Navbar

Il CSS distingue desktop e mobile, centra il brand, limita la ricerca e definisce avatar e dropdown. Bootstrap decide i breakpoint tramite classi `d-none`, `d-lg-grid` e `d-lg-none`; il CSS rifinisce la composizione.

Riferimenti: `static/assets/css/style.css:25-146`, `templates/base.html:16-117`.

19.3 Form e upload

I pannelli raggruppano semanticamente lingue, schedule e foto. Le griglie foto hanno dimensioni stabili, nomi lunghi con ellissi e stato visivo di rimozione.

Riferimenti: `static/assets/css/style.css:231-367`, `templates/create_tour.html:16-124`.

19.4 Homepage, card e profili

Le sezioni usano classi condivise per etichette e titoli. Card tour e profili hanno layout dedicati e stati vuoti. Le media query trasformano azioni e gruppi da righe a colonne sui display stretti.

Riferimenti: `static/assets/css/style.css:147-230`, `static/assets/css/style.css:368-471`, `static/assets/css/style.css:507-544`.

19.5 Pagina Tours

`tours.css` è caricato soltanto nella lista tour. Gestisce accordion filtri, immagini, overlay, pannello hover e varianti mobile/desktop.

Riferimenti: `templates/tours.html:5-7`, `static/assets/css/tours.css:1-165`.

19.6 Dettaglio tour

`tour-detail.css` è caricato soltanto nel dettaglio. Gestisce hero, galleria, modal, booking agenda, avatar e responsive behavior.

Riferimenti: `templates/tour_detail.html:4-6`, `static/assets/css/tour-detail.css:1-301`.

20. Tracce complete da saper raccontare

20.1 Dalla navbar alla ricerca

- 1 Il form in `base.html` invia `q` con GET.
- 2 La route `tours` legge `request.args`.
- 3 `filtered_tours` confronta il testo con titolo e tema.
- 4 La route passa risultati e filtri a `tours.html`.
- 5 Jinja genera le card o lo stato vuoto.

Riferimenti: `templates/base.html:22-27`, `app.py:599-620`, `app.py:400-431`, `templates/tours.html:37-108`.

20.2 Dalla registrazione al login

- 1 Il browser valida campi HTML e pannello lingue.
- 2 Flask ripete le validazioni.
- 3 La password viene hashata.
- 4 SQLite garantisce email unica.
- 5 Dopo il commit l'utente viene mandato al login.
- 6 Il login verifica ruolo e hash e apre la sessione.

Riferimenti: `templates/register.html:12-88`, `app.py:658-715`, `schema.sql:13-23`, `app.py:625-655`.

20.3 Dalla pianificazione al dettaglio

- 1 La guida apre una route protetta.
- 2 Compila campi, schedule, tappe e foto.
- 3 JS dà feedback, Flask valida davvero.
- 4 L'agenda guida viene controllata.
- 5 Tour e dettagli vengono inseriti nella stessa transazione.
- 6 Il redirect apre il dettaglio appena creato.

Riferimenti: `app.py:727-778`, `app.py:436-582`, `templates/create_tour.html:1-278`, `app.py:914-945`.

20.4 Dal click Book al profilo

- 1 Il dettaglio mostra date future generate dallo schedule.
- 2 Il partecipante sceglie data, persone e nomi.
- 3 Il POST ricontrolla data, schedule, capienza e overlap.
- 4 SQLite inserisce o riattiva la prenotazione.
- 5 Il profilo rilegge prenotazioni e calcola la cancellabilità.

Riferimenti: `app.py:359-382`, `templates/tour_detail.html:156-220`, `app.py:977-1065`, `app.py:1107-1142`.

20.5 Dalla cancellazione alla nuova disponibilità

- 1 Il profilo mostra Cancel se mancano almeno 24 ore.
- 2 Il POST ripete ownership, stato e soglia.
- 3 Lo stato diventa `cancelled`, la riga resta nel DB.
- 4 `active_reserved_places` considera solo booked.
- 5 La data mostra nuovamente i posti liberati.

Riferimenti: `app.py:1131-1140`, `app.py:1145-1183`, `app.py:288-301`.

20.6 Dal tour passato al report

- 1 Il profilo guida raggruppa le prenotazioni per data.
- 2 Calcola se la partenza è passata e se manca il report.
- 3 Il form limita gli effettivi agli attesi.

- 4 Il POST ripete tutti i controlli e salva la foto.
- 5 Il vincolo unico chiude definitivamente il report per quella data.

Riferimenti: `app.py:1209-1252`, `templates/guide_profile.html:44-116`, `app.py:1260-1317`,
`schema.sql:100-109`.

21. Domande insidiose e risposte oneste

Perché `sqlite3.Row`?

Permette `row["email"]` invece di indici numerici fragili. Migliora la leggibilità senza introdurre un ORM. Riferimento: `db.py:9-13`.

Perché niente ORM?

SQL esplicito rende visibili JOIN, aggregazioni e vincoli richiesti dal corso. Un ORM sarebbe utile in un progetto più grande, ma aggiungerebbe un livello da imparare e spiegare.

Perché alcuni filtri sono in Python?

Per mantenere leggibile un dataset piccolo e riusare `enrich_tour`. Su scala maggiore si sposterebbero ricerca, aggregazioni e filtri in una query SQL unica.

Perché frontend e backend duplicano alcune regole?

Il frontend offre feedback prima dell'invio; il backend è la fonte autorevole e non può essere aggirato modificando HTML o JavaScript.

Perché password hash e non cifratura reversibile?

Il server deve verificare una password, non recuperarla. L'hash con salt riduce il danno se il DB viene letto. Riferimenti: `app.py:703`, `app.py:643`.

Perché una guida non può prenotare?

Il ruolo rappresenta un contesto operativo unico. La UI lo comunica e il backend lo impone. Per partecipare servirebbe un account participant con un'altra email, dato il requisito di unicità globale.

Perché l'editing è tutto bloccato?

Cambiare durata, schedule, lingua, capienza o tappe dopo una prenotazione attiva altererebbe l'impegno accettato. La soluzione scelta è semplice e coerente; una più avanzata richiederebbe istanze per singola data e gestione delle eccezioni.

Le prenotazioni passate bloccano l'edit?

Sì, se lo stato è ancora booked, perché l'helper non filtra per data. Solo lo stato cancelled non blocca. Riferimento: `app.py:304-313`.

Perché non esiste una tabella delle singole uscite?

Il modello genera le date da uno schedule settimanale. È sufficiente per il requisito scelto. Una tabella `tour_occurrences` sarebbe necessaria per spostare o annullare una sola domenica senza cambiare l'intero schedule.

Come viene impedito il doppio booking?

Controllo applicativo più `UNIQUE(user_id, tour_id, tour_date)`. Una riga cancellata viene riattivata invece di duplicata.

È presente CSRF protection?

Non è presente un token CSRF dedicato. In una versione di produzione si potrebbe aggiungere Flask-WTF o token propri a tutti i POST. Il progetto resta sulle tecnologie e sul perimetro affrontati, ma è importante saper riconoscere il limite.

La validazione email è completa?

Il browser usa `type="email"`; il backend richiede `@` e `.` e normalizza il valore. È una validazione intenzionalmente semplice. Una verifica RFC completa o email di conferma sarebbe un'estensione.

Perché debug=True?

Serve nello sviluppo locale. Il deploy WSGI di PythonAnywhere importa l'app e non esegue il blocco `__main__`; in produzione il debugger non deve essere esposto.

22. Matrice di copertura completa

Questa matrice assegna **ogni riga** dei sorgenti a un blocco concettuale. Gli intervalli sono adiacenti e coprono dal numero 1 all'ultima riga del file.

22.1 app.py - 1.321 righe

Righe	Blocco
1-13	Import standard, Flask, Flask-Login, SQLite, Werkzeug e connessione DB
14-47	Configurazione app, upload, costanti e LoginManager
48-63	Modello sessione User e lingue parlate
64-80	User loader Flask-Login
81-91	Context processor per costanti Jinja
92-97	Sezione validation e normalizzazione email
98-115	Validazione e salvataggio upload
116-127	Eliminazione sicura upload
128-139	Parsing interi con limiti
140-165	Conversioni orarie e overlap
166-185	Parsing date, datetime e nomi
186-201	Redirect interno sicuro e controllo ruolo
202-220	Lettura schedule e schedule per data
221-246	Formattazione schedule, tappe e foto
247-272	Conteggi like/commenti e stato like utente
273-285	Lettura tour con JOIN guida
286-315	Posti, disponibilità e blocco editing
316-335	Overlap agenda guida
336-358	Overlap agenda partecipante
359-384	Generazione date future prenotabili
385-399	Arricchimento della riga tour
400-433	Lettura e filtraggio dei tour
434-455	Parsing schedule dal form
456-527	Validazione completa form tour
528-553	File foto e parsing id rimozione
554-565	Reindicizzazione posizioni foto
566-584	Inserimento schedule, tappe e foto
585-598	Route homepage e Top 3
599-622	Route Tours e filtri GET
623-657	Route login
658-717	Route registrazione
718-724	Route logout
725-780	Route creazione tour
781-911	Route modifica tour
912-947	Route dettaglio tour
948-976	Route like toggle
977-1067	Route prenotazione
1068-1096	Route commento
1097-1106	Route di smistamento profilo
1107-1144	Route profilo partecipante

Righe	Blocco
1145-1185	Route cancellazione prenotazione
1186-1257	Route profilo guida
1258-1319	Route invio report
1320-1321	Avvio server di sviluppo

22.2 db.py - 26 righe

Righe	Blocco
1-8	Import e percorso assoluto del database
9-15	Connessione, row factory e foreign key
16-24	Inizializzazione database da schema
25-26	Esecuzione reset solo come script diretto

22.3 schema.sql - 109 righe

Righe	Blocco
1-12	Foreign key e DROP in ordine dipendenze
13-24	Tabella users
25-38	Tabella tours
39-47	Tabella tour_schedule
48-55	Tabella tour_stops
56-64	Tabella tour_photos
65-79	Tabella reservations
80-89	Tabella tour_likes
90-99	Tabella comments
100-109	Tabella tour_reports

22.4 requirements.txt - 3 righe

Righe	Blocco
1-3	Dipendenze Python runtime

22.5 templates/base.html - 151 righe

Righe	Blocco
1-14	Documento, metadata, favicon, CSS e blocchi Jinja
15-18	Body e apertura header/navbar
19-70	Navbar desktop, ricerca, ruolo e autenticazione
71-117	Navbar mobile e menu collapse
118-131	Rendering dei messaggi flash
132-133	Blocco contenuto delle pagine
134-147	Footer, disclaimer e attribuzioni
148-151	Bootstrap JS, script specifici e chiusure

22.6 templates/index.html - 102 righe

Righe	Blocco
1-3	Ereditarietà e apertura content
4-20	Introduzione homepage
21-59	Card How it works
60-100	Top 3 dinamica e stato vuoto
101-102	Chiusura main e blocco

22.7 templates/tours.html - 113 righe

Righe	Blocco
1-8	Ereditarietà, titolo e CSS specifico
9-25	Breadcrumb e intestazione
26-78	Sidebar accordion e form filtri
79-110	Griglia tour, dettagli e stato vuoto
111-113	Chiusure struttura e blocco

22.8 templates/login.html - 41 righe

Righe	Blocco
1-4	Ereditarietà, titolo e content
5-11	Layout e intestazione login
12-33	Form email, password, ruolo e submit
34-41	Link registrazione e chiusure

22.9 templates/register.html - 88 righe

Righe	Blocco
1-4	Ereditarietà, titolo e content
5-11	Layout e intestazione registrazione
12-44	Dati account, ruolo e nota email unica
45-59	Pannello lingue guida
60-70	Submit, link login e chiusure
71-88	JavaScript visibilità/validazione lingue

22.10 templates/create_tour.html - 278 righe

Righe	Blocco
1-4	Ereditarietà e titolo create/edit
5-16	Layout, intestazione e requisiti
17-54	Form e campi principali
55-72	Schedule settimanale
73-83	Tappe e descrizione
84-119	Foto esistenti e upload multiplo
120-129	Submit e chiusura contenuto
130-147	JavaScript schedule
148-175	JavaScript tappe minime
176-204	Stato e pulsanti foto correnti
205-211	Ricostruzione FileList con DataTransfer
212-258	Elenco file e validazione totale foto
259-278	Eventi Remove/Undo e inizializzazione

22.11 templates/tour_detail.html - 351 righe

Righe	Blocco
1-7	Ereditarietà, titolo e CSS specifico
8-17	Breadcrumb
18-50	Testata, like e controllo edit
51-67	Hero, foto e meeting point
68-100	Descrizione, dati e tappe
101-111	Miniature galleria
112-155	Form e lista commenti
156-222	Booking card per stato/ruolo
223-264	Modal e carousel fotografie
265-283	JavaScript apertura foto selezionata
284-317	JavaScript validazione nomi ospiti
318-351	JavaScript posti, selezione e listener

22.12 templates/participant_profile.html - 71 righe

Righe	Blocco
1-4	Ereditarietà, titolo e content
5-13	Intestazione profilo
14-44	Ciclo prenotazioni e informazioni
45-60	Stato e azione cancellazione
61-71	Stato vuoto e chiusure

22.13 templates/guide_profile.html - 128 righe

Righe	Blocco
1-4	Ereditarietà, titolo e content
5-21	Intestazione e Plan tour
22-43	Ciclo tour e azioni Open/Edit
44-63	Gruppi prenotazioni e stato report
64-85	Tabella partecipanti
86-95	Riepilogo report già inviato
96-113	Form report disponibile
114-128	Stati vuoti e chiusure

22.14 static/assets/css/style.css - 544 righe

Righe	Blocco
1-9	Variabili di palette e ombra
10-24	Body flex, sfondo, font e link
25-146	Navbar desktop/mobile, ricerca, avatar e dropdown
147-206	Homepage, titoli, card informative e breadcrumb
207-230	Regole mobile per intro, ricerca e brand
231-276	Form auth, pannelli e schedule
277-367	Upload, lista file, foto correnti e form card
368-414	Card Top 3 e stato vuoto
415-471	Profili, azioni, gruppi data e pre-wrap
472-506	Footer e autore
507-544	Responsive di schedule, foto, profili e footer

22.15 static/assets/css/tours.css - 165 righe

Righe	Blocco
1-18	Tipografia della pagina Tours
19-40	Accordion dei filtri
41-74	Card, immagine, hover e overlay
75-105	Caption, badge e titolo
106-133	Pannello dettagli e focus filtri
134-159	Variante mobile
160-165	Variante desktop larga

22.16 static/assets/css/tour-detail.css - 301 righe

Righe	Blocco
1-17	Hero e immagine principale
18-44	Trigger zoom e hint
45-75	Gradiente e overlay meeting point
76-94	Contenuto, info grid e tappe
95-123	Galleria miniature e focus
124-179	Modal, carousel, frecce e contatore
180-188	Booking sticky e pulsante like
189-248	Agenda, date, stati selected/full e testi
249-266	Avatar e cinque tonalità
267-280	Tipografia contenuti/commenti
281-301	Responsive hero, gallery, booking e modal

23. Checklist finale per la preparazione

Prima dell'orale bisogna saper fare senza leggere:

- disegnare le relazioni principali del database;
- spiegare una richiesta GET e una POST complete;
- distinguere sessione, ruolo e ownership;
- spiegare perché una prenotazione è legata a una data reale;
- ricostruire la formula degli overlap;
- spiegare il doppio controllo applicazione/database;
- spiegare commit, rollback e query parametrizzate;
- indicare perché Jinja non è HTML statico;
- indicare cosa fa JavaScript e perché non è una difesa;
- spiegare differenza fra cancellazione logica e DELETE;
- riconoscere limiti realistici: CSRF, validazione file, filtro in Python, schedule senza eccezioni per singola data e debug locale;
- seguire la matrice e descrivere qualunque intervallo scelto dal professore.

La risposta migliore non è “questa riga fa una query”, ma:

Questa query serve a una regola precisa del flusso, usa parametri per separare SQL e input, restituisce dati che la route trasforma per il template, e il relativo vincolo nel database impedisce uno stato incoerente.